

ODESA I. I. MECHNIKOV NATIONAL UNIVERSITY
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATION TECHNOLOGIES
DEPARTMENT OF OPTIMAL CONTROL AND ECONOMIC CYBERNETICS

Foundations of Blockchain Technology

Essay

The Bin Packing Problem as an NP-complete Problem and its Applications in Blockchain

Uliana Hrab

Contents

1	Introduction. Basic definitions of complexity theory	2
1.1	The deterministic Turing machine	2
1.2	The nondeterministic Turing machine	2
1.3	Complexity classes. The classes P and NP	2
1.4	Polynomial-time reduction	3
1.5	NP-hard and NP-complete problems	3
2	Statement of the bin packing problem	3
3	Proof that bin packing is NP-complete	4
3.1	Membership in NP	4
3.2	NP-hardness: reduction from PARTITION	4
3.3	Conclusion	5
4	Hardness of approximation	5
5	Applications of the problem in blockchain	5
5.1	Block construction: packing transactions under the block limit	6
5.2	Rollups and batching transactions into blobs (L2 scaling)	6
5.3	Sharding and load distribution across nodes and shards	6
5.4	The DA layer and storage	6
6	Solution methods used in practice	6
6.1	Exact methods	7
6.2	Approximation algorithms and heuristics (with guarantees)	7
6.3	Asymptotic schemes and metaheuristics	7
6.4	What this looks like in blockchain	7
7	Conclusions	8

1 Introduction. Basic definitions of complexity theory

1.1 The deterministic Turing machine

A **Turing machine (TM)** is an abstract mathematical model of a computing device that formalises the intuitive notion of an “algorithm”. Formally, a deterministic Turing machine is a seven-tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

where Q is a finite set of states; Σ is the input alphabet; $\Gamma \supseteq \Sigma$ is the tape alphabet (containing the blank symbol); $q_0 \in Q$ is the initial state; $q_{\text{accept}}, q_{\text{reject}}$ are the accepting and rejecting states; and

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

is the **transition function**. The machine has a tape, infinite in both directions and divided into cells, and a read/write head. At each step, being in state q and reading the symbol a , the machine determines unambiguously (hence *deterministic*) via δ : which state to move to, which symbol to write into the current cell and where to move the head (left L or right R).

The key property: for every configuration there is **exactly one** next step. A computation of a deterministic TM is therefore a single, uniquely determined sequence of configurations.

1.2 The nondeterministic Turing machine

A **nondeterministic Turing machine (NTM)** differs in that the transition function is replaced by a transition **relation**:

$$\delta \subseteq (Q \times \Gamma) \times (Q \times \Gamma \times \{L, R\}),$$

that is, from a single configuration **several** admissible next steps may exist. A computation of an NTM is no longer a linear sequence but a **computation tree**: at each branching point the machine may “choose” any of the permitted transitions.

The accepted convention is this: an NTM **accepts** an input word if **there exists at least one** branch of the computation tree that ends in the state q_{accept} . It is convenient to read nondeterminism as the machine’s ability to “guess” the right option (a certificate) and then verify it deterministically. This is not a physically realisable model but a mathematical tool for classifying problems.

1.3 Complexity classes. The classes P and NP

We consider **decision problems** — problems with a “yes”/“no” answer. Complexity is measured as a function of the input length n in the worst case.

The class P (Polynomial time) is the set of decision problems solvable by a **deterministic** TM in polynomial time $O(n^k)$ for some constant k . Informally these are the “efficiently (quickly) solvable” problems: sorting, shortest paths (Dijkstra), primality testing (AKS) and so on.

The class NP (Nondeterministic Polynomial time) is the set of problems solvable by a **non-deterministic** TM in polynomial time. An equivalent (and more practical) definition goes via **certificate verification**: a problem belongs to NP if for every “yes” input x there exists a **certificate (witness)** y of polynomial length such that a deterministic TM can **verify** the pair (x, y) in polynomial time and confirm the “yes” answer.

Example: for the bin packing problem the certificate is a concrete assignment of items to bins. Finding such an assignment is hard, but **checking** a proposed one is easy.

Clearly $P \subseteq NP$ (if a problem can be solved in polynomial time, no certificate is needed). Whether $P = NP$ holds is the central open problem of theoretical computer science (one of the Millennium Prize Problems). The prevailing conjecture is $P \neq NP$.

1.4 Polynomial-time reduction

A problem A is **polynomial-time reducible** to a problem B (written $A \leq_p B$) if there exists a function f , computable in polynomial time, that transforms every input x of A into an input $f(x)$ of B such that

$$x \text{ is a "yes" instance of } A \iff f(x) \text{ is a "yes" instance of } B.$$

The meaning: if we can solve B efficiently then we can solve A too (transform the input and apply the solver for B). Hence B is “no easier” than A . Reduction is transitive: $A \leq_p B$ and $B \leq_p C \Rightarrow A \leq_p C$.

1.5 NP-hard and NP-complete problems

A problem B is called **NP-hard** if **every** problem in NP is polynomial-time reducible to it: $\forall A \in NP : A \leq_p B$. An NP-hard problem is “no easier” than any problem in NP (it need not itself belong to NP).

A problem B is called **NP-complete** if it simultaneously:

1. belongs to the class **NP** ($B \in NP$), and
2. is **NP-hard**.

NP-complete problems are the “hardest” problems in NP. Their key property: if even one NP-complete problem can be solved in polynomial time then $P = NP$, and **all** problems in NP become polynomially solvable. The first problem proved NP-complete was Boolean satisfiability (the **Cook–Levin theorem**, 1971). Richard Karp (1972) then reduced SAT to 21 further problems, establishing the standard method of proving NP-completeness **by reduction from an already known NP-complete problem**.

Scheme for proving that a problem B is NP-complete:

- (a) show that $B \in NP$ (exhibit a polynomial-time certificate verifier);
- (b) choose a known NP-complete problem A and construct a polynomial-time reduction $A \leq_p B$.

It is exactly this scheme that we will use for the bin packing problem.

2 Statement of the bin packing problem

Informally. We are given a collection of items of various “sizes” and an unlimited supply of identical bins of fixed capacity. All items must be placed into bins so as to use **as few bins as possible** (the sum of the item sizes in each bin must not exceed its capacity).

Optimisation formulation. Given:

- a finite set of items $U = \{1, 2, \dots, n\}$;
- the size of each item $s_i \in (0, 1]$ (normalised) or a positive integer s_i ;
- the bin capacity C (for the normalised version $C = 1$).

Find a partition of the set U into a minimum number of subsets (bins) $B_1, \dots, B_m$ such that for each j

$$\sum_{i \in B_j} s_i \leq C.$$

The value m^* (the minimum number of bins) is the optimum of the problem.

Formulation as a decision problem (for complexity theory).

BIN PACKING (decision).

Input: items with integer sizes $s_1, \dots, s_n$, a bin capacity C and an integer K .

Question: can all the items be placed into **at most** K bins of capacity C (so that the total size of the items in each bin does not exceed C)?

It is this version (“are K bins enough?”) that is regarded as NP-complete.

3 Proof that bin packing is NP-complete

We proceed by the standard scheme: (1) BIN PACKING $\in NP$; (2) PARTITION $\leq_p$ BIN PACKING.

3.1 Membership in NP

The **certificate** is an assignment function $a : \{1, \dots, n\} \rightarrow \{1, \dots, K\}$ mapping each item to the index of a bin. The length of the certificate is $O(n \log K)$, that is, polynomial in the size of the input.

A **verifier** running in polynomial time: for each bin $j \in \{1, \dots, K\}$ it computes the sum $\sum_{i: a(i)=j} s_i$ and checks that it does not exceed C . That is $O(n)$ additions and K comparisons in total. If all bins satisfy the constraint, the answer is “yes”. So the existence of a valid packing is verifiable in polynomial time, and therefore **BIN PACKING** $\in NP$.

3.2 NP-hardness: reduction from PARTITION

As the base problem we take the classical NP-complete **PARTITION** problem:

PARTITION.

Input: a multiset of positive integers $A = \{a_1, \dots, a_n\}$ with even sum $S = \sum a_i = 2T$.

Question: does there exist a subset $A' \subseteq A$ with sum $\sum_{a \in A'} a = T$ (that is, can A be split into two parts with equal sums T and T)?

(The NP-completeness of PARTITION is a well-known fact; it in turn reduces from SUBSET SUM, which comes from Karp’s chain of reductions starting at 3-SAT.)

Construction of the reduction f . Given a PARTITION instance with numbers $a_1, \dots, a_n$ and sum $S = 2T$, we build a BIN PACKING instance as follows:

- items with sizes $s_i = a_i$ for $i = 1, \dots, n$;

- bin capacity $C = T$;
- number of bins $K = 2$.

The transformation is obviously carried out in polynomial (in fact linear) time.

Correctness of the reduction. We show that PARTITION answers “yes” $\Leftrightarrow$ the constructed BIN PACKING instance answers “yes”.

($\Rightarrow$) Suppose a partition $A = A' \cup A''$ with $\sum A' = \sum A'' = T$ exists. Put all items of A' into the first bin and those of A'' into the second. The load of each is exactly $T = C \leq C$. Two bins therefore suffice — BIN PACKING answers “yes”.

($\Leftarrow$) Suppose the items have been packed into $K = 2$ bins of capacity $C = T$. Then the combined load of the two bins does not exceed $2C = 2T$. But the total size of all items equals $S = 2T$. Hence both inequalities “load $\leq T$ ” must become **equalities**: the load of each bin is exactly T . The item sets of these two bins give the required partition of the numbers into two parts with sums T and T — PARTITION answers “yes”.

Thus f correctly reduces PARTITION to BIN PACKING: $\text{PARTITION} \leq_p \text{BIN PACKING}$. Since PARTITION is NP-complete, BIN PACKING is **NP-hard**.

3.3 Conclusion

BIN PACKING $\in NP$ (Section 3.1) and BIN PACKING is NP-hard (Section 3.2) $\Rightarrow$ **the bin packing problem is NP-complete.** ■

It follows that the optimisation (minimisation) version of the problem is **NP-hard** as well: if we could find the minimum number of bins m^* in polynomial time, then comparing $m^* \leq K$ would immediately settle the decision version.

4 Hardness of approximation

An interesting and practically important consequence of the reduction in Section 3.2 is that the bin packing problem is hard to solve **even approximately**.

A packing into 2 bins in the constructed instance is possible $\Leftrightarrow$ a partition of the numbers into two equal halves exists. Therefore any polynomial-time algorithm that could reliably distinguish the case “optimum = 2” from “optimum = 3” would solve the NP-complete problem PARTITION. Hence, **assuming $P \neq NP$, there is no polynomial-time algorithm with an approximation ratio better than $3/2$** (otherwise it would distinguish 2 from 3).

This does not contradict the existence of good algorithms: the $3/2$ bound concerns the **absolute** worst case. In the **asymptotic** sense (when the optimum is large) the problem is approximated very well — an asymptotic polynomial-time approximation scheme is known for it (APTAS, Fernandez de la Vega and Lueker, 1981), as is the Karmarkar–Karp algorithm, which uses OPT plus only $O(\log^2 \text{OPT})$ bins.

5 Applications of the problem in blockchain

The bin packing problem arises wherever a **resource of fixed capacity** must be filled with a set of **discrete units** with minimal “waste”. In blockchain there are many such places.

5.1 Block construction: packing transactions under the block limit

A block has a hard upper bound on its “size”: in Bitcoin this is the block weight (≈ 4 million weight units), in Ethereum the block gas limit (≈ 30 million gas). Each transaction “weighs” a certain number of bytes or units of gas. The validator’s (miner’s) task is to fill the block with transactions without exceeding the limit.

- If the goal is to **maximise the total fee in a single block** subject to the limit, this is the **knapsack problem**.
- If instead one has to **distribute the whole pool of pending transactions over the minimum number of blocks** (for example when planning the filling of several upcoming blocks, or when batching), this is already the **bin packing problem**: transactions are the items, the block limit is the bin capacity, and a block is a bin.

5.2 Rollups and batching transactions into blobs (L2 scaling)

In layer-two rollup solutions (optimistic and ZK rollups) hundreds of transactions are **batched** and published to L1 in a single message of limited size. Since EIP-4844, data is published in **blobs** of fixed size (≈ 128 KB each). Distributing a set of compressed transactions over the **minimum number of blobs** is bin packing directly. Since each blob is paid for, minimising their number reduces the L2 cost to users directly.

5.3 Sharding and load distribution across nodes and shards

When state (accounts, contracts, data) is distributed across shards or storage nodes, the capacity of each shard is bounded (memory, throughput). Placing objects into the minimum number of shards without overloading any of them is a variant of bin packing (a related formulation is multiprocessor scheduling / makespan with a fixed number of “bins”).

5.4 The DA layer and storage

At the data availability layer and in decentralised storage (IPFS/Filecoin, Arweave) data fragments are laid out into sectors or blocks of fixed size — again, so as to minimise the number of storage units occupied and the “waste” of capacity.

The common denominator: **a bounded “bin” (block, blob, shard, sector) + discrete units (transactions, data) + the aim of minimising the number of bins and the waste** — that is the bin packing problem.

6 Solution methods used in practice

Since the problem is NP-hard, searching for the exact optimum on real (large) input sizes is impractical. In practice **exact methods** (for small and medium instances) are combined with **fast heuristics and approximation algorithms** (for online workloads such as real-time block construction).

6.1 Exact methods

- **Integer linear programming (ILP)** with branch-and-bound / branch-and-cut solvers.
- **Column generation (branch-and-price)** based on the Gilmore–Gomory formulation; today this is the most powerful exact approach for classical bin packing.
- **Dynamic programming** — only for small n or small capacities (because it is pseudo-polynomial).

6.2 Approximation algorithms and heuristics (with guarantees)

These run fast (usually $O(n \log n)$) and return a solution guaranteed to be close to the optimum:

- **Next Fit (NF)** — puts the item into the current bin; if it does not fit, opens a new one and “forgets” the previous one. Guarantee: $NF \leq 2 \cdot OPT$.
- **First Fit (FF)** — puts the item into the **first** bin in which it fits. Guarantee: $FF \leq 1.7 \cdot OPT$ (asymptotically).
- **Best Fit (BF)** — puts the item into the bin with the **smallest remaining free space** in which it still fits. Asymptotic guarantee as for FF ($\approx 1.7 \cdot OPT$).
- **First Fit Decreasing (FFD)** — first **sorts** the items by decreasing size, then applies First Fit. The classical result: $FFD \leq \frac{11}{9} OPT + \frac{6}{9}$. One of the best simple heuristics.
- **Best Fit Decreasing (BFD)** — likewise, with Best Fit.

The split into regimes:

- **Offline** (all items known in advance) — sorting is possible: FFD, BFD.
- **Online** (items arrive one at a time and decisions are irrevocable) — NF, FF, BF; this is precisely the regime that corresponds to building a block “on the fly” from the mempool.

6.3 Asymptotic schemes and metaheuristics

- **APTAS** (Fernandez de la Vega and Lueker) and the **Karmarkar–Karp algorithm** — theoretically near-optimal when the optimum is large.
- **Metaheuristics** — genetic algorithms, simulated annealing, ant colony and tabu search — for complex industrial variants (with additional constraints: item compatibility, multidimensional sizes and so on).

6.4 What this looks like in blockchain

In real clients, building a block is an **online greedy** process under a resource limit: transactions from the mempool are usually ordered by “price per unit” (fee per unit of gas, that is by profitability) and then added greedily to the block while space remains — a practical blend of knapsack ideas (maximising the fee) and bin packing (filling up to the limit). In rollup sequencers the batcher applies exactly First Fit / Best Fit style heuristics to distribute compressed transactions over blobs, minimising their number and hence the cost of publishing data to L1.

7 Conclusions

1. Bin packing is a classical problem of combinatorial optimisation: place items of given sizes into the minimum number of bins of fixed capacity.
2. Its decision version is **NP-complete**: membership in NP is proved by exhibiting a polynomial-time verifier (checking the assignment of items to bins), and NP-hardness by a simple and transparent polynomial-time reduction from PARTITION ($C = T$, $K = 2$).
3. The same reduction shows that the problem is hard **even to approximate**: assuming $P \neq NP$ there is no polynomial-time algorithm with a ratio better than $3/2$ (in the worst case), although asymptotically very good schemes do exist.
4. In blockchain the problem arises naturally in **block construction** under a size or gas limit, **batching rollup transactions into blobs**, **sharding** and **data storage**.
5. In practice exact methods (ILP, column generation) for small instances are combined with fast heuristics carrying guarantees (First Fit, Best Fit, First Fit Decreasing) for online workloads, which matches how blocks are actually built and how rollup sequencers work.

References

- [1] Garey M. R., Johnson D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. — W. H. Freeman, 1979. (The PARTITION and BIN PACKING problems, [SR1], [SR6].)
- [2] Karp R. M. *Reducibility Among Combinatorial Problems* // Complexity of Computer Computations. — 1972. — P. 85–103.
- [3] Cook S. A. *The Complexity of Theorem-Proving Procedures* // STOC. — 1971. — P. 151–158.
- [4] Sipser M. *Introduction to the Theory of Computation*. — 3rd ed. — Cengage, 2013. (Turing machines, the classes P and NP, NP-completeness.)
- [5] Cormen T., Leiserson C., Rivest R., Stein C. *Introduction to Algorithms*. — 3rd ed. — MIT Press, 2009. (Approximation algorithms, bin packing heuristics.)
- [6] Fernandez de la Vega W., Lueker G. S. *Bin packing can be solved within $1 + \varepsilon$ in linear time* // Combinatorica. — 1981.
- [7] Karmarkar N., Karp R. M. *An efficient approximation scheme for the one-dimensional bin-packing problem* // FOCS. — 1982.
- [8] Buterin V. et al. *Ethereum Yellow Paper* (the block gas limit); EIP-1559, EIP-4844 (fee mechanics and data blobs).